

Linux Cold Storage Archival Suite (LCSAS)

Definitive Architecture and Implementation Guide

Version: 1.0

Target Media: Blu-ray (BD-R), M-Disc, LTO Tape

Core Philosophy: Write-Once, Read-Never (until restore), Holographic Indexing.

1. Executive Summary

This document defines the architecture for a petabyte-scale, offline-first archival system designed for Linux environments. The system addresses specific limitations of traditional backup tools when applied to Write-Once-Read-Many (WORM) optical media.

Key Capabilities:

1. **Infinite Incrementalism:** Utilizing **Content-Defined Chunking (CDC)** to ensure that file moves, renames, and duplicates consume zero additional storage payload on optical media.
 2. **Multi-Tenant Isolation:** Safely mixing distinct datasets (e.g., "Family", "Work") encrypted with different keys on the same physical disc without compromising cryptographic isolation.
 3. **Holographic Indexing:** Embedding the complete archive catalog and repository metadata onto *every* physical volume. Any surviving disc can bootstrap a full recovery of the archive.
 4. **Local Mirror Strategy:** Maintaining a permanent "Hot" local tier to facilitate instant media consolidation (merging small incrementals into large M-Discs) without physically retrieving offsite volumes.
 5. **Bit-Rot Immunity:** Integrating **DVDDisaster (RS03)** to wrap data in error-correction codes capable of recovering from physical media decay or scratches.
-

2. Theoretical Architecture

2.1 The Data Engine: Content-Defined Chunking (CDC)

Traditional incremental backups rely on timestamps or file paths. If a 10GB folder is renamed, traditional tools treat it as 10GB of new data, forcing a rewrite to the optical disc.

The Solution: We utilize **Rustic** (a high-performance Rust implementation of the Restic format).

- **Mechanism:** Rustic scans files using a rolling hash. It cuts the file into blobs (chunks) based on content boundaries, not file boundaries.
- **Result:** If a file is moved or renamed, the chunk hashes remain identical. The system detects these existing chunks in the index and writes **zero** payload to the optical disc, only a small metadata reference. This solves the "rename explosion" problem on WORM media.

2.2 Storage Topology: The "Split-Horizon" Model

To manage offline data, we decouple the *logical* repository from the *physical* storage.

Tier	Name	Storage Type	Role	Content
Tier 1	Local Mirror	HDD/SSD	Hot / Read-Write	A complete copy of all data packs and metadata. Used for fast deduplication, restores, and generating compilations.
Tier 2	Staging Area	NVMe / SSD	Transient	Temporary buffer where new, un-burned packs are collected and organized into ISO structures before burning.
Tier 3	Cold Archive	BD-R / M-Disc	Cold / WORM	The offline disaster recovery copy. Stored offsite.

2.3 The Catalog: Decentralized Tracking

Since rustic expects a single cohesive repository, and our discs are fragmented, we require an external index to map logical data to physical locations.

We utilize a **SQLite Database** (archive_master.db) as the "Hypervisor."

- It tracks which **Packs** are on which **Volumes**.
 - It supports **Multi-Copy Tracking**: The DB knows that Pack_A exists on BD_001 (local), M_DISC_005 (offsite), and LTO_001 (vault).
 - **Holographic Strategy**: A copy of this SQLite database is written to the root of every optical disc. If you lose the server, the latest disc contains the map to the entire fleet.
-

3. Database Schema Design (SQLite)

The following schema is designed to track terabytes of data across thousands of discs while allowing for volume deprecation (pruning) and redundancy tracking.

SQL

-- 1. TRACKS PHYSICAL MEDIA (Discs, Tapes)

```
CREATE TABLE volumes (  
  volume_id INTEGER PRIMARY KEY AUTOINCREMENT,  
  label TEXT UNIQUE NOT NULL,      -- Human label: "LCSAS_BD_2024_01"  
  uuid TEXT UNIQUE NOT NULL,      -- Machine ID: "550e8400-e29b..."  
  media_type TEXT,                -- "BD25", "MDISC100", "LTO8"  
  location TEXT,                  -- "Offsite_Bank_Vault", "Home_Shelf"  
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
  status TEXT DEFAULT 'CLOSED'    -- "OPEN" (Staging), "CLOSED", "DESTROYED"  
);
```

-- 2. TRACKS LOGICAL DATA (Encrypted Chunks)

```
CREATE TABLE packs (  
  pack_id INTEGER PRIMARY KEY AUTOINCREMENT,  
  sha256 TEXT UNIQUE NOT NULL,    -- The Rustic Pack Filename  
  size_bytes INTEGER NOT NULL,  
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

-- 3. THE MAP (Many-to-Many Relationship)

-- Crucial: This allows a single pack to exist on multiple discs (Redundancy)

```
CREATE TABLE volume_packs (  
  volume_id INTEGER,  
  pack_id INTEGER,  
  PRIMARY KEY (volume_id, pack_id),  
  FOREIGN KEY(volume_id) REFERENCES volumes(volume_id),  
  FOREIGN KEY(pack_id) REFERENCES packs(pack_id)  
);
```

-- 4. REPOSITORY METADATA (For Multi-Tenant Management)

```
CREATE TABLE repositories (  
  repo_id TEXT PRIMARY KEY,      -- Rustic Repository ID  
  name TEXT,                     -- "Family_Photos", "Work_Docs"  
  encryption_key_id TEXT        -- ID of the key used (for user prompt)
```

);

4. Implementation Workflow

4.1 Ingest: The Daily Mirror

Goal: Backup data to the Local Mirror (Tier 1) and secure it.

1. **Execution:** Run rustic backup targeting the Local Mirror path (e.g., /mnt/mirror/family).
2. **Isolation:** Run separate backup commands for each dataset (Family, Work) using their specific encryption keys. They can target separate folders in the Local Mirror (/mnt/mirror/family, /mnt/mirror/work) to maintain strict separation.
3. **Result:** The Local Mirror contains all new data. Deduplication happens immediately against this local copy.

4.2 Processing: The Burn Cycle

Goal: Identify new data on the Mirror that hasn't been archived yet and burn it.

1. **Delta Analysis:**

The Orchestrator script queries the DB:

SQL

```
SELECT sha256 FROM packs
WHERE pack_id NOT IN (
    SELECT pack_id FROM volume_packs
    JOIN volumes ON volume_packs.volume_id = volumes.volume_id
    WHERE volumes.location = 'Offsite' -- Optional: Filter by specific tier
)
```

Result: A list of pack files present on the Local Mirror that have never been written to an offsite disc.

2. **Bin Packing:**

The script selects packs from this list until their total size reaches the media capacity (e.g., 23GB for a 25GB BD-R). It prioritizes older packs first.

3. **Staging:**

- Create a temp directory: /mnt/staging/ISO_ROOT/
- **Hardlink** the selected packs from Local Mirror to /mnt/staging/ISO_ROOT/data/. (Hardlinking is instant and takes no space).
- **Inject Holographic Metadata:** Copy the *entire* snapshots/, index/, and keys/ directories from the Local Mirror to /mnt/staging/ISO_ROOT/metadata/.
- **Inject Database:** Copy the current archive_master.db to /mnt/staging/ISO_ROOT/.

4. **Burning (Xorriso + DVDIsaster):**

- Generate ISO image.

- Run dvd disaster to augment the ISO with error correction codes (RS03).
- Burn to Disc.
- 5. **Commit:**
 - On successful verify, update the SQLite DB:
 1. Create new Volume entry.
 2. Insert rows into volume_packs linking the burned Pack IDs to the new Volume ID.

4.3 Consolidation: The Migration Strategy

Goal: Merge 5 small BD-Rs into 1 large M-Disc without reading the old discs.

1. **Identify Sources:** You want to consolidate volumes BD_001 to BD_005.
2. **Query Active Data:**
 - Run rustic prune --dry-run against the Local Mirror. This tells you which packs are "dead" (deleted files) and can be discarded.
 - Query the DB for all packs belonging to BD_001...BD_005.
 - **Intersect:** Packs_To_Burn = (Packs on BD_001..005) AND (Packs NOT marked for Prune).
3. **Fetch from Mirror:**

The script retrieves these specific Packs_To_Burn from the **Local Mirror** (Tier 1). It does *not* ask you to insert the old BD-Rs.
4. **Burn M-Disc:**

Write the filtered data stream to the 100GB M-Disc.
5. **Update DB:**
 - Create Volume MDISC_001.
 - Link packs to MDISC_001.
 - Mark BD_001..BD_005 as DEPRECATED or DESTROYED in the database.

5. Software Stack and Scripts

You will need a custom Python script ("The Orchestrator") to glue these components together.

Required Tools:

- **Rustic:** The backup engine.
- **Python 3:** For the Orchestrator logic.
- **Xorriso:** For ISO mastering and burning.
- **DVD disaster:** For ECC generation.
- **Smartmontools/hdparm:** For drive health checking (optional).

5.1 Orchestrator Script Snippet (Python)

This pseudo-code demonstrates the logic for identifying un-backed-up packs and preparing the staging area.

Python

```
import sqlite3
import os
import subprocess

DB_PATH = "/var/lib/lcsas/archive.db"
MIRROR_PATH = "/mnt/local_mirror_hdd"
STAGING_PATH = "/mnt/nvme_staging"

def get_unarchived_packs(conn):
    """Finds packs in Mirror that are not in the DB's volume_packs table"""
    # 1. Get all packs physically present in Mirror
    local_packs = set(os.listdir(f"{MIRROR_PATH}/data"))

    # 2. Get all packs recorded in DB
    cursor = conn.cursor()
    cursor.execute("SELECT sha256 FROM packs")
    recorded_packs = set([r for r in cursor.fetchall()])

    # 3. Identify completely new packs (not even in DB yet)
    new_packs = local_packs - recorded_packs

    # 4. Register new packs into DB (but not assigned to volume yet)
    for p in new_packs:
        size = os.path.getsize(f"{MIRROR_PATH}/data/{p}")
        cursor.execute("INSERT INTO packs (sha256, size_bytes) VALUES (?,?)", (p, size))
    conn.commit()

    # 5. Query for packs not assigned to any volume
    cursor.execute("""
        SELECT sha256, size_bytes FROM packs
        WHERE pack_id NOT IN (SELECT pack_id FROM volume_packs)
    """)
    return cursor.fetchall()

def stage_for_burn(packs_to_burn):
    os.makedirs(f"{STAGING_PATH}/data", exist_ok=True)

    for pack_name, size in packs_to_burn:
        src = f"{MIRROR_PATH}/data/{pack_name}"
        dst = f"{STAGING_PATH}/data/{pack_name}"
```

```
# Use Hardlink to save space/time
os.link(src, dst)
```

```
# Copy Holographic Metadata (Indexes, Snapshots) from ALL repos
# Note: Copying small files is fast.
subprocess.run(f"cp -r {MIRROR_PATH}/index {STAGING_PATH}/", shell=True)
subprocess.run(f"cp -r {MIRROR_PATH}/snapshots {STAGING_PATH}/", shell=True)
subprocess.run(f"cp -r {MIRROR_PATH}/keys {STAGING_PATH}/", shell=True)

# Dump DB
subprocess.run(f"cp {DB_PATH} {STAGING_PATH}/catalog.db", shell=True)
```

6. Restoration Process (Disaster Recovery)

When the Local Mirror is lost, and you only have the discs:

1. **Bootstrap:** Insert the **Most Recent Disc** you have.
2. **Load Catalog:** Copy catalog.db from the disc to your temp recovery machine.
3. **Identify Restore Set:**
 - Run `rustic restore <snapshot> --dry-run --json`.
 - This outputs a list of required Pack IDs.
4. **Generate Pick List:**
 - Run a query against catalog.db:
SQL
`SELECT v.label, v.location
FROM volumes v
JOIN volume_packs vp ON v.volume_id = vp.volume_id
JOIN packs p ON vp.pack_id = p.pack_id
WHERE p.sha256 IN (...list of required packs...)
GROUP BY v.label`
5. **Data Fetch:** The system tells you: "Please insert Disc 4 and Disc 9."
6. **Verify:** You copy the packs from those discs to a local temp folder.
7. **Decrypt:** Rustic restores your files from the temp folder using the keys (which are also on the discs).

7. Recommendations

1. **Directory Structure:** Keep your Local Mirror organized by repository.
 - `/mnt/mirror/repo_family/`
 - `/mnt/mirror/repo_work/`

- The orchestrator script should scan *all* these data directories and treat them as a single pool of packs to be burned.
- 2. **Encryption:** Do not encrypt the optical disc itself (e.g., LUKS on BD-R). Rely on Rustic's object-level encryption. This prevents a scratch on the LUKS header from destroying the whole disc.
- 3. **Redundancy:** Because you track "Copies" in the DB, you can easily script a job to "Ensure every pack exists on at least 2 volumes." The script will find packs that only exist on 1 volume and queue them for the next burn.
- 4. **LTO Future:** This schema supports LTO tapes perfectly. You just add LTO as a `media_type` in the volumes table. The workflow is identical, just writing to `/dev/st0` instead of `/dev/sr0`.

This architecture provides an enterprise-grade, cryptographically secure, and physically resilient archive system using commodity hardware.